

Capstone Project Executive Summary

Gerry Jr. Agluba

June 3, 2026

1 Introduction

Service delivery and lifecycle management in modern cloud-native environments currently imposes a critical bottleneck dictated by **cognitive load paradox** and **fragility of static pipeline automation**. Standard pipelines often fail to adapt to unpredictable real-world events resulting in slow incident triage and inconsistent remediation. Research reveals a severe **5:3 failure ratio** between pre-merge safety checks and post-merge reality [1]. **70% of Engineers** have reported **active burn-out** and on-call stress on incident response tasks [2]. Economically, the cost of this operational gap is staggering: for high-stakes industries like Telecom, a single hour of unresolved downtime can translate into **\$1.4 million in financial exposure** [3]. To overcome these limitations, modern platforms requires an **intelligent orchestration system** that is *conversational, extensible and autonomous*. This work introduced **MASON (Multi-Agent Harness for Service Orchestration and Optimization)**, an intelligent orchestration layer utilizing a multi-agent system designed for autonomous executions.

2 System Architecture and Operations

2.1 Core Orchestration and Intelligence Layer

2.1.1 Task Routing

Using built-in **Language-Based Task Routing Policy**, Non-adhoc tasks are further routed to specialized service orchestration and optimization agents for task execution and reporting through Kafka-based message queues [4]. Specialized agents has accessed to developed orchestration and eventing tools to orchestrates workloads over a live Kubernetes environment.

2.1.2 Agent Management

Repetitive Tasks can be transformed into a dynamic event-triggered agentic workflows. Mason enforces an extensible and declarative framework where agent behaviors, scopes and tools are defined. This enables declarative reasoning and seamless lifecycle control—including agent registration, instantiation, and runtime termination.

2.1.3 Reasoning Model and Memory Management

The architecture hosts Small Language Models (SLMs) running on specialized, high-performance inference backends running on localized hardware utilizing a single commodity *NVIDIA 5060 TI* GPU:

- (Setup A) **Qwen 3 Instruct 4B**[9] served via a **TensorRT-LLM**[11] inference server and;
- (Setup B) **Qwen 3.5 4B** served via a **vLLM**[10] inference server.

Persistence and context preservations are managed via two memory servers: **Redis Agent Memory Server**[8] that manages ongoing agent conversational state; and **Redis Server** [7] that manages task-related results, summary; tool calls and observations; evaluations as well as agent states.

2.2 Unified Operational Interface

The primary interaction layer is an IDE - based conversational and operational interface engineered for platform and operations teams.

3 Evaluation Methodology

The validation of the agent harness is conducted using an end-to-end evaluation pipeline.

1. **Test Scenario Curation.** Test scenarios are curated for two main agentic use cases with standard execution structures containing diverse pre-conditions and execution parameters. **Deployment Scenarios** - covers general application delivery workflows with standard, constrained and various environment policy conditions. **Ad-Hoc Tasks** - covers unstructured and generic tasks including service diagnostics, resource optimization, workload analysis, policy friction; and incident diagnosis and recovery.
2. **Automated Execution.** Automated scripts are developed to execute evaluation process: environment setup with pre-condition, task event triggering, and test teardown. Real-time agent execution and results, performance logs and other telemetries are collected. Other system metrics collected also includes: **Time-to-first-token (TTFT)**, **End-to-End Workflow Latency** and **Total Token Usage**.
3. **LLM-as-a-Judge Framework**[12]. To ensure valid and stringent performance validation; task summaries, tool calls and results, and execution paths were evaluated via a batch API multi-judge framework. Evaluations were validated using two separate and independent foundational models: **gemini-3.1-flash-lite-preview**[14] and **gpt-5.4**[13] with the following defined agent metrics: **Task Completion**, **Tool Accuracy**, **Step Efficiency**, **Plan Adherence**, and **Faithfulness**.

4 Results and Discussion

High Performance. The MASON framework demonstrated strong operational efficacy across all evaluated environments, achieving a **peak task completion rate of 92% on complex deployment workflows** and maintaining a **stable 88% completion rate on unstructured ad-hoc tasks** under rigorous multi-judge cross-examination.

Precision vs. Operational Efficiency. A clear performance trade-off was observed between the model generations. Setup B (Qwen 3.5 4B) consistently scored $\sim 80\%$ across all qualitative metrics (alignment, faithfulness, and plan adherence), emphasizing the inherent operational necessity of the model’s native “thinking mode” when navigating dense, interdependent cloud failure paths. Conversely, Setup A (Qwen 3 Instruct 4B) suffered from logical hallucinations, dropping to scores between 60% and 70%. However, the older model established massive resource efficiencies for predictable, high-frequency routines, successfully slashing workflow latency by 66%, reducing total token consumption costs by 80.6%, and requiring 42% fewer tool invocation overheads.

Fast Evolutionary Trajectory. The closed-loop optimization layer proved highly effective at accelerating system adaptation. By analyzing intercepting error logs, historical reflections, and judge critiques, the Prompt-Optimizer agent dynamically refined its own execution parameters. This continuous behavioral refinement loop successfully accelerated ad-hoc task pass rates from an unoptimized from **52% up to 76% in only three sequential optimization generations**.

Tables 1 and 2 details the performance test results.

5 Conclusion

MASON’s self-optimizing orchestration layer provides a precise and low-cost solution for managing complex cloud-native environments. Additionally, it eliminates tool fragmentation and minimizes human intervention for resilient, cost-effective, and autonomous cloud operations.

Table 1: Task Completion Rates Across Evaluation Ecosystems

Task Type	Gemini Evaluation		GPT Evaluation	
	Setup A	Setup B	Setup A	Setup B
Deployment Tasks	80.0%	92.0%	72.0%	81.0%
Ad-Hoc Tasks	78.0%	88.0%	72.0%	88.0%

Table 2: Comprehensive Agent Metrics Evaluations (GPT 5.4)

Metrics	Ad-Hoc Tasks		Deployment Tasks	
	Setup A	Setup B	Setup A	Setup B
<i>Agent Quality Metrics</i>				
Tool Accuracy	0.70	0.80	0.74	0.81
Step Efficiency	0.71	0.79	0.69	0.82
Plan Adherence	0.71	0.81	0.63	0.78
Faithfulness	0.67	0.79	0.62	0.71
<i>Operational Efficiency</i>				
Workflow Latency (s)	31.4986	93.6711	140.1829	217.4672
Function / Tool Calls	4.88	6.72	7.88	13.60
Generated Token Cost (tokens)	297.60	1,540.16	994.80	2,915.76
Time-to-First-Token (TTFT - s)	4.0248	2.6683	2.8923	1.6507

References

- [1] S. Sun, D. Friberg, and M. Staron. “Good” and “Bad” failures in industrial CI/CD – balancing cost and quality assurance. *Chalmers University of Technology and University of Gothenburg*, June 2024. Available at: <https://www.researchgate.net/publication/390845943>.
- [2] Catchpoint SRE Team. *Catchpoint Site Reliability Engineering (SRE) Report 2025*. Technical Industry Survey, Catchpoint Systems, 2025. Available at: <https://www.sherlocks.ai/blog/traditional-sre-vs-modern-sre-what-every-engineering-leader-needs-to-know-in-2026>.
- [3] IrisAgent. *AI for MTTR Reduction: How to Cut Resolution Times with Intelligent Automation*. Technical Report, IrisAgent, Inc., April 2026. Available at: <https://irisagent.com/blog/ai-for-mttr-reduction-how-to-cut-resolution-times-with-intelligent/>.
- [4] Apache Software Foundation. (2026). *Apache Kafka: A Distributed Streaming Platform*. Available at: <https://kafka.apache.org/> (Accessed: May 2026).
- [5] Agentic AI Foundation. (2026). *Model Context Protocol (MCP) Specification: Standardizing AI Assistant Integration with External Tools and Data Sources*. Available at: <https://modelcontextprotocol.io/> (Accessed: May 2026).
- [6] Containers Project Community. (2026). *Kubernetes MCP Server: Native Go-Based Model Context Protocol Server for Kubernetes Cluster Operations*. Available at: <https://github.com/kubernetes/mcp-server> (Accessed: May 2026).
- [7] Redis Software. (2026). *Redis: An In-Memory Data Structure Store used as a Database, Cache, and Message Broker*. Available at: <https://redis.io/> (Accessed: May 2026).
- [8] Redis Developers. (2025). *Managing LLM Bot and Multi-Agent State Context with In-Memory Redis Vector and Key-Value Stores*. Available at: <https://redis.io/solutions/vector-search/> (Accessed: May 2026).
- [9] Alibaba Cloud. (2025). *Qwen3 Series Technical Report: Advancing Small Language Models for Localized Agent Tasks*. Available at: <https://github.com/QwenLM/Qwen3> (Accessed: May 2026).

- [10] K. Kwon, M. Li, S. Zhuang, J. Sheng, L. Zheng, C. Yu, J. E. Gonzalez, I. Stoica, and I. Zhang. (2023). “Efficient Memory Management for Large Language Model Serving with PagedAttention,” in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP ’23)*, pp. 611–626.
- [11] NVIDIA Corporation. (2024). *NVIDIA TensorRT-LLM: An Open-Source Library for Optimizing Large Language Model Inference*. Available at: <https://github.com/NVIDIA/TensorRT-LLM> (Accessed: May 2026).
- [12] M. Desmond, Z. Ashktorab, Q. Pan, C. Dugan, and J. M. Johnson. (2024). “EvaluLLM: LLM assisted evaluation of generative outputs,” in *Companion Proceedings of the 29th International Conference on Intelligent User Interfaces (IUI ’24 Companion)*, Association for Computing Machinery, New York, NY, USA, pp. 30–32. <https://doi.org/10.1145/3640544.3645216>.
- [13] OpenAI. (2026). *The GPT-5.4 Foundation Model Architecture and Advanced Reasoning Capability*. Available at: <https://developers.openai.com/api/docs/models/gpt-5.4> (Accessed: May 2026).
- [14] Google DeepMind. (2026). *Gemini 3.1 Flash-Lite: Optimizing High-Speed, Low-Cost Batch Inference and Context-Thrifty Evaluation*. Available at: <https://docs.cloud.google.com/gemini-enterprise-agent-platform/models/gemini/3-1-flash-lite> (Accessed: May 2026).
- [15] O. A. Gbenga. *A Research Report on the Evolution and Indispensability of Platform Engineering*. Platform Engineering Review, February 2025. Available at: <https://medium.com/@engr gbenga/a-research-report-on-the-evolution-and-indispensability-of-platform-engineering-8535f4d6577c>.
- [16] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, editors. *Site Reliability Engineering: How Google Runs Production Systems*. O’Reilly Media, 1st edition, 2016.
- [17] Growin. *Kubernetes Resilience Checklist (2025): How to Build Resilient Backends with Kubernetes*. Growin Engineering Blog, September 2025. Available at: <https://www.growin.com/blog/resilient-backends-kubernetes-2025/>.
- [18] M. Koneru and R. Pappula. Cascading failure modes in Model-as-a-Service architectures: When your dependencies think. *International Journal of Applied Engineering & Technology*, 7(2):104–112, June 2025.
- [19] Cloud Native Computing Foundation (CNCF). *CNCF Platform Engineering Survey 2025*. CNCF Research Reports, January 2025. Available at: <https://platformengineering.org/blog/announcing-the-state-of-platform-engineering-vol-4>.